

Smart Product Recommendation Engine

with Review Sentiment Boost

Collaborative Filtering · NLP Sentiment Analysis · Real-time Re-ranking

AI-Powered · Sentiment-Aware · Full-Stack

Version	Status	Owner	Last Updated
v1.0	Draft → Review	Kishan Bharghav V (SRM CSE)	March 2026

Stack: Python · Scikit-learn · HuggingFace Transformers · SQLite · Streamlit

1. Executive Summary

Most recommendation systems answer one question: "What do other users with similar taste like?" They optimize for popularity and correlation — and miss a critical signal hiding in plain sight: what people actually say about the products.

The Smart Product Recommendation Engine with Review Sentiment Boost changes this. It combines classic collaborative filtering to surface what users are likely to enjoy, then applies a real-time NLP sentiment pipeline over product reviews to re-rank those results — demoting products that users dislike in their own words, even if the star rating looks acceptable.

The result is a recommendation layer that is not just statistically accurate but qualitatively honest. Bad products cannot hide behind volume. The system is deployed as an interactive Streamlit app backed by SQLite, and the full ML pipeline is reproducible, explainable, and extendable.

Attribute	Detail
Product Name	SentimentRec — Smart Product Recommendation Engine
Core Innovation	Sentiment-boosted re-ranking on top of collaborative filtering
Primary Users	Shoppers; internal team for admin & analytics
Platform	Web app (Streamlit), local or cloud deployment
Data Store	SQLite — users, products, ratings, reviews, sentiment scores
ML Stack	Scikit-learn (SVD/CF), HuggingFace (DistilBERT sentiment)
MVP Timeline	8 weeks

2. Problem Statement

2.1 The Gap in Existing Recommendation Systems

Collaborative filtering and matrix factorization are the backbone of most recommender systems. They work by identifying patterns in user-item interaction data — ratings, clicks, purchases — and projecting similar users onto a shared preference space. This is powerful, but it has a fundamental blind spot.

A product rated 3.8 stars by 2,000 users carries significant collaborative weight. But if 600 of those reviews say things like "broke after two weeks," "misleading description," or "never buying again,"

the star average has washed out the signal. The recommendation engine, seeing only the numerical rating, will keep surfacing that product.

2.2 Why This Matters

- Users lose trust in recommendations when suggested products disappoint them in ways that "everyone else" seemed to notice.
- Businesses carry reputational risk by amplifying low-quality products through their own recommendation layer.
- Review text is one of the richest feedback signals available — and almost no lightweight rec system uses it at ranking time.

2.3 The Opportunity

By parsing the sentiment of review text and feeding a continuous sentiment score into the ranking function, we can create a system that is more aligned with true user satisfaction — not just historical co-occurrence patterns. Products with strong negative sentiment get demoted regardless of popularity. Products with consistently positive reviews get a ranking boost.

3. Goals & Non-Goals

3.1 Goals

#	Goal	Success Metric
G1	Implement collaborative filtering using SVD to generate baseline recommendations	Precision@10 $\geq$ 0.35 on held-out test set
G2	Run HuggingFace sentiment pipeline on all product reviews and persist scores	Sentiment classification accuracy $\geq$ 88%
G3	Re-rank CF output using a tunable sentiment boost formula	Negative-sentiment products demoted in $\geq$ 90% of test cases
G4	Persist all data in SQLite (users, products, ratings, reviews, scores)	Schema stable; no data loss under concurrent reads
G5	Deliver a Streamlit UI with personalized recommendations and explanation	User can see sentiment score and review snippet per product
G6	Admin dashboard showing model performance and sentiment distribution	Charts render with live SQLite data on page refresh

3.2 Non-Goals (v1.0)

- Real-time model retraining on new ratings (batch nightly retraining is sufficient for MVP).

- Multi-modal input (images, video reviews) — text reviews only.
- Multi-language sentiment analysis — English reviews only in v1.
- Deep learning-based collaborative filtering (neural CF) — classical SVD is intentional for interpretability.
- Mobile application or REST API exposure — Streamlit web UI only.

4. User Personas

	Priya — Shopper	Arjun — Power User	Admin / Analyst
Goal	Find products she'll actually like	Explore recommendations & tweak sentiment weight	Monitor model health & review quality
Pain	Wastes money on hyped but low-quality items	Black-box recs with no explanation	No visibility into what the model is doing
Needs	Clear, ranked list with sentiment indicator	Adjustable sentiment boost slider	Dashboard with Precision, Recall, and sentiment breakdown
Tech	Non-technical	Technically curious	Data-literate, not ML expert

5. System Architecture

5.1 High-Level Data Flow

The system follows a four-stage pipeline. Each stage is independently testable and the outputs of each stage are persisted in SQLite so the next stage can be run independently.

① Data Ingestion	② CF Model Training	③ Sentiment Scoring	④ Boosted Re-ranking
Load users, products, ratings, reviews into SQLite tables	Train SVD model on user-item rating matrix; store latent factors	Run DistilBERT over every review; write sentiment_score to DB	Merge CF score + weighted sentiment score; return ranked list
SQLite: users, products, ratings, reviews	Scikit-learn SVD / Surprise library	HuggingFace pipeline('sentiment-analysis')	$\text{final_score} = \alpha \cdot \text{cf_score} + (1-\alpha) \cdot \text{sentiment_score}$

5.2 Ranking Formula

The core re-ranking formula is:

$$\text{final_score}(u, i) = \alpha \cdot \text{cf_score}(u, i) + (1 - \alpha) \cdot \text{sentiment_score}(i)$$

Symbol	Range	Description
cf_score(u, i)	[0, 1] norm.	SVD-predicted rating for user u on item i, normalized
sentiment_score(i)	[0, 1]	Aggregate positive sentiment ratio across all reviews for item i
α (alpha)	[0.0 – 1.0]	Tunable weight; default 0.7. Exposed as UI slider for power users
final_score(u, i)	[0, 1]	Combined score used for final product ranking

6. Database Schema (SQLite)

All persistent state is managed in a single SQLite database file. The schema is designed to support both the ML pipeline and the Streamlit UI with zero external dependencies.

Table: users

Column	Type	Description
user_id	INTEGER PRIMARY KEY	Auto-incremented unique identifier
username	TEXT NOT NULL UNIQUE	Display name
email	TEXT NOT NULL UNIQUE	User email
created_at	DATETIME DEFAULT NOW	Account creation timestamp

Table: products

Column	Type	Description
product_id	INTEGER PRIMARY KEY	Auto-incremented unique identifier

name	TEXT NOT NULL	Product name
category	TEXT	Product category / tag
description	TEXT	Product description
avg_rating	REAL	Computed average star rating (cached)
created_at	DATETIME	Listing date

Table: ratings

Column	Type	Description
rating_id	INTEGER PRIMARY KEY	Auto-incremented
user_id	INTEGER (FK → users)	User who rated
product_id	INTEGER (FK → products)	Product rated
rating	REAL NOT NULL	Star rating (1.0 – 5.0)
rated_at	DATETIME	When the rating was submitted

Table: reviews

Column	Type	Description
review_id	INTEGER PRIMARY KEY	Auto-incremented
user_id	INTEGER (FK → users)	Author of the review
product_id	INTEGER (FK → products)	Reviewed product
review_text	TEXT NOT NULL	Raw review body (English)
reviewed_at	DATETIME	Submission timestamp

Table: sentiment_scores

Column	Type	Description
score_id	INTEGER PRIMARY KEY	Auto-incremented
review_id	INTEGER (FK → reviews)	Associated review
product_id	INTEGER (FK → products)	Denormalized for fast aggregation
sentiment_label	TEXT	POSITIVE / NEGATIVE / NEUTRAL

confidence	REAL	Model confidence score [0, 1]
scored_at	DATETIME	When the sentiment model ran

7. Functional Requirements

7.1 Recommendation Engine

- FR-01: The system SHALL generate a ranked list of top-N product recommendations for any registered user.
- FR-02: The CF model SHALL use Singular Value Decomposition (SVD) on the user-product rating matrix.
- FR-03: CF scores SHALL be normalized to [0, 1] before blending with sentiment scores.
- FR-04: Final ranking SHALL apply the formula: $\text{final_score} = \alpha \cdot \text{cf_score} + (1-\alpha) \cdot \text{sentiment_score}$.
- FR-05: Products with a mean confidence-weighted sentiment score below 0.35 SHALL be flagged as "Sentiment Warning" regardless of final_score.

7.2 Sentiment Analysis Pipeline

- FR-06: The system SHALL process all review texts using the HuggingFace 'distilbert-base-uncased-finetuned-sst-2-english' model.
- FR-07: Each review SHALL produce a sentiment_label (POSITIVE/NEGATIVE) and a confidence score.
- FR-08: Per-product sentiment_score SHALL be computed as the ratio of POSITIVE-labeled reviews weighted by confidence.
- FR-09: Sentiment scores SHALL be recomputed nightly (or on-demand via admin panel).
- FR-10: The pipeline SHALL handle empty reviews or null text gracefully without crashing.

7.3 User Interface (Streamlit)

- FR-11: A login/register page SHALL authenticate users against the SQLite users table.
- FR-12: The recommendations page SHALL display top-10 products with: name, category, final_score, sentiment badge, and one representative review snippet.
- FR-13: A sidebar slider SHALL allow users to adjust the α (alpha) blending weight in real time.
- FR-14: Clicking a product SHALL open a detail view with all reviews and their individual sentiment labels.
- FR-15: An Admin page (password-protected) SHALL display: Precision@10, RMSE, sentiment label distribution chart, and worst-scoring products.

7.4 Data Management

- FR-16: The SQLite schema SHALL enforce foreign key constraints (FK integrity on ratings, reviews, sentiment_scores).

- FR-17: The system SHALL support seeding the database from a CSV file (e.g., Amazon product reviews dataset).
- FR-18: Duplicate ratings (same user_id, product_id) SHALL update the existing record, not insert a new one.

8. Non-Functional Requirements

ID	Category	Requirement	Target
NFR-01	Performance	Recommendation list load time for a returning user	< 2 seconds (pre-scored)
NFR-02	Performance	Sentiment batch processing for 10,000 reviews	< 15 minutes on CPU
NFR-03	Scalability	SQLite database size ceiling for MVP	Up to 500K reviews
NFR-04	Reliability	App uptime during demo/presentation window	99% (no background jobs)
NFR-05	Accuracy	SVD RMSE on held-out 20% test set	≤ 0.85 on 1–5 scale
NFR-06	Accuracy	Sentiment model classification accuracy	≥ 88% on SST-2 benchmark
NFR-07	Usability	First-time user can get recommendations with no training	Task completion < 60s
NFR-08	Maintainability	Each module (CF, sentiment, DB, UI) independently importable	All modules pass unit tests
NFR-09	Security	Admin panel access	Password hash check (SHA-256)
NFR-10	Reproducibility	Model outputs are reproducible	Seeded random_state throughout

9. Feature Breakdown & Prioritization

Feature	Priority	Effort	Sprint	Owner
SQLite schema + seed script	P0 — Must	Low	1	Backend
SVD collaborative filtering model	P0 — Must	Medium	2	ML
HuggingFace sentiment pipeline	P0 — Must	Medium	2	ML

Score persistence in sentiment_scores	P0 — Must	Low	3	Backend
Boosted re-ranking formula	P0 — Must	Low	3	ML
Streamlit login / register	P0 — Must	Low	4	Frontend
Recommendations page with sentiment UI	P0 — Must	Medium	4	Frontend
Alpha slider (real-time re-rank)	P1 — Should	Low	5	Frontend
Product detail + review sentiment view	P1 — Should	Low	5	Frontend
Admin dashboard (metrics + charts)	P1 — Should	Medium	6	Frontend
Nightly retraining cron script	P2 — Nice	Medium	7	Backend
Export recommendations as CSV	P2 — Nice	Low	8	Frontend

10. Technical Stack Decisions

Layer	Technology	Rationale
CF / ML	Scikit-learn / Surprise	SVD is well-understood, fast, and produces normalized predictions. Surprise wraps it cleanly for RecSys tasks.
Sentiment NLP	HuggingFace Transformers	distilbert-sst2 is 40% smaller than BERT with 97% of accuracy — ideal for CPU-bound batch scoring on laptops.
Database	SQLite	Zero-config, file-based, perfect for portfolio demos. Single source of truth for pipeline stages. Easily swappable to PostgreSQL.
UI / Frontend	Streamlit	Python-native, no HTML/JS overhead. Excellent for ML demos with reactive widgets (slider for alpha blending).
Language	Python 3.10+	Ecosystem alignment — all chosen libraries have first-class Python support.
Serialization	joblib / pickle	Persist trained SVD model between sessions without retraining on each app load.

11. Edge Cases & Failure Modes

Scenario	Risk	Mitigation
New user — no ratings (cold start)	CF cannot generate recommendations	Fall back to top products by sentiment_score alone
Product has no reviews	sentiment_score is undefined	Default sentiment_score = 0.5 (neutral); flag in UI
All reviews are positive (review bombing)	Artificially inflated sentiment_score	Cap sentiment weight contribution; add review count floor (≥ 5 reviews)
HuggingFace model unavailable / timeout	Sentiment pipeline fails	Serve stale scores from last successful run; log error
Malformed review text (HTML entities, emojis)	Tokenizer errors or garbage sentiment output	Sanitize text before inference; fallback to neutral if exception
$\alpha = 0$ (pure sentiment only)	No personalization — same list for all users	Show warning in UI; suggest minimum $\alpha \geq 0.3$
Very sparse rating matrix	SVD underfit; poor CF predictions	Log RMSE and warn if < 50 ratings per product in training

12. Project Milestones & Timeline

Sprint	Week	Deliverable	Done When
S1	Week 1–2	Database design finalized, schema created, seed data loaded from CSV	SQLite file exists; queries return data
S2	Week 2–3	SVD CF model trained, RMSE logged, model serialized to disk	model.pkl loads and generates predictions
S3	Week 3–4	Sentiment pipeline runs on all reviews, scores written to sentiment_scores table	All rows in reviews have a matching score
S4	Week 4–5	Re-ranking formula implemented and unit-tested with mock data	Test cases pass for demotion of negative products
S5	Week 5–6	Streamlit app: login, registration, recommendations page with sentiment badges	End-to-end demo runnable
S6	Week 6–7	Alpha slider, product detail page, admin dashboard with charts	All P1 features delivered
S7	Week 7–8	Testing, documentation, README, requirements.txt, GitHub push	Repo is demo-ready; README explains setup

13. Success Metrics

13.1 ML Model Quality

- SVD RMSE ≤ 0.85 on 20% hold-out test set.
- Sentiment classification accuracy $\geq 88\%$ on labeled review sample.
- Precision@10 ≥ 0.35 — at least 3–4 of the top-10 recommendations match implicit test preference.
- Negative-review products ranked in bottom 30% after sentiment boost in $\geq 90\%$ of test simulations.

13.2 Product & UX Quality

- Recommendations page loads in < 2 seconds for users with pre-scored products.
- Admin dashboard renders with live SQLite data with no manual refresh workaround.
- A first-time user can register, see personalized recommendations, and adjust sentiment weight within 60 seconds.
- GitHub repo has README with one-command setup (`pip install -r requirements.txt && streamlit run app.py`).

13.3 Portfolio Signal

- The project demonstrates three distinct competencies in one codebase: relational data modeling (SQLite), classical ML (SVD), and NLP (HuggingFace Transformers).
- Sentiment-boosted re-ranking is an original contribution not typically found in tutorial-level rec systems.
- Streamlit deployment makes it immediately demoable without Docker or cloud infra setup.

14. Open Questions

#	Question	Owner / Decision
Q1	Should we use the Surprise library's SVD implementation or build from scratch with NumPy for more control?	Kishan — decide by end of Sprint 1
Q2	Which public dataset to seed with? (Amazon Reviews, MovieLens + fake products, or synthetic data?)	Kishan — before schema finalization
Q3	Should we pre-compute sentiment scores offline or run them lazily per product on first view?	Decide based on dataset size in Sprint 2
Q4	What is the minimum review count threshold before sentiment_score is considered reliable?	Experiment in Sprint 3; default: 5 reviews

Q5	How should the admin dashboard authenticate? Plain password in config or hashed in DB?	Use SHA-256 stored in config.py for MVP
-----------	--	---

15. Appendix

A. Glossary

Term	Definition
Collaborative Filtering	A recommendation technique that predicts a user's preference by finding patterns among many users' historical behavior.
SVD (Singular Value Decomposition)	A matrix factorization technique that decomposes the user-item rating matrix into latent factor vectors.
Sentiment Score	A numerical value [0, 1] representing the proportion of positive sentiment across a product's review set, weighted by model confidence.
Alpha (α)	The tunable blending weight that controls the balance between CF score and sentiment score in the final ranking formula.
Precision@10	The fraction of the top-10 recommended items that are actually relevant to the user — a standard RecSys evaluation metric.
RMSE	Root Mean Squared Error — measures how far predicted ratings deviate from actual ratings on the test set.
Sentiment Warning	A UI label applied to products whose sentiment_score falls below 0.35, regardless of their CF popularity.
Cold Start	The inability of a CF model to make recommendations for a new user or item with no historical interaction data.

B. Recommended Dataset Sources

- Amazon Product Reviews (UCSD) — juliaang.org/datasets or HuggingFace datasets hub.
- MovieLens 100K — can be adapted with synthetic review text for demonstration purposes.
- Synthetic generation using Faker + random rating distributions is acceptable for MVP if a public dataset presents licensing concerns.

C. Future Roadmap (v2+)

- Neural Collaborative Filtering (NCF) — replace SVD with an embedding-based neural model for improved cold-start handling.
- Aspect-Based Sentiment Analysis — extract specific aspects (quality, shipping, packaging) from reviews for granular product scoring.
- Multi-language support — add language detection and multilingual sentiment models.

- REST API layer (FastAPI) — expose recommendations as an API for integration with any frontend.
- Containerization — Docker + docker-compose for one-click deployment.

— *End of Document* —